SongFinder

Project Report for *Signal, Image and Video*

Matteo Morellini, Riccardo Benevelli

Abstract

SongFinder is a course project on automatic music identification. We compare a landmark-based Shazam-style pipeline based on spectrogram peak constellations and combinatorial hashing, with GraFP (GraFPrint), a learned fingerprinting method based on GNNs and contrastive learning. We report Top-1 recognition accuracy, query latency, and an entropy-based confidence metric under clean audio, additive noise at different SNRs (using background noises from GraFP augmentations), and convolutional reverberation, across query durations of 3–10 s.

1 Introduction

Automatic audio identification aims to match a short query excerpt to a reference track in a database, even under realistic recording conditions. This report presents SongFinder, a unified framework that implements and benchmarks two representative fingerprinting paradigms: a classical landmark-based method (Shazam-style) and the GraFP neural embedding approach. Our goal is to quantify the trade-offs between accuracy, robustness, and latency under controlled degradations (noise and reverberation) and varying query durations.

2 Background

Landmark-based fingerprinting (Shazam-style). Given a reference track, the audio is converted into a time–frequency representation (STFT spectrogram) and reduced to a sparse set of salient local maxima (a *constellation map*). In our implementation, the front-end is controlled by three hyperparameters: *target_sr* (target sample rate, which sets the maximum representable frequency by Nyquist's theorem), n_fft (FFT window size, which controls frequency resolution), and *hop_length* (stride between consecutive frames). Following the logarithmic nature of human hearing, we split the spectrum into six bands (e.g., the first band spans $[1, 10]$ and the last spans $[161, 511]$) and, for each time frame and band, we select the frequency bin with the highest amplitude as a peak. Fingerprints are built by selecting an "anchor" peak and pairing it with peaks in a forward target zone; the number of pairs per anchor is governed by the fan_out hyperparameter. Each pair defines a reproducible token (hash) that encodes $(f_1, f_2, \Delta t)$, i.e., anchor frequency, target frequency, and their time separation. All tokens extracted from the database are stored in an inverted index

$$\texttt{hash} \rightarrow \texttt{list[(songID, anchorTime)]}.$$

At query time, tokens from the query excerpt retrieve candidate matches from the index. For each match, an offset $\Delta T = T_{db} - T_q$ is computed; a correct match yields many tokens agreeing on the same offset. The final score for a candidate song is the height of the dominant peak in the offset histogram (temporal-consistency voting), and the best-scoring song is returned.

Neural fingerprinting with GraFP (GraFPrint). GraFP replaces hand-crafted hashes with a learned embedding for short audio segments. An audio segment is represented as a log-mel spectrogram; positional information (time and frequency indices) is concatenated to preserve the geometry of time–frequency points. A convolutional front-end produces a compact latent map, which is flattened into N nodes; a k-NN graph is constructed dynamically in the latent space, and graph convolutions refine node features via message passing. The segment-level fingerprint is obtained by pooling node embeddings and projecting to a low-dimensional vector (e.g., 128-D).

The encoder is trained with contrastive learning (NT-Xent): two augmented views of the same segment (cropping, noise mixing, filtering, reverb) form a positive pair, while other segments in the batch act as negatives. For identification, all reference embeddings are stored in a vector index (FAISS) to enable approximate nearest-neighbour (ANN) search. Given a query excerpt, embeddings are computed for overlapping windows, nearest neighbours are retrieved for each window, and candidates are aggregated (e.g., by vote counts and/or offset compensation across windows) to produce a final track prediction.

3 Data and Preprocessing

Audio tracks are converted to mono and resampled to a common sampling rate (16 kHz). For each track we compute time–frequency representations.

Queries are generated by taking crops of different durations (3 s, 5 s, 10 s) and applying controlled degradations:

- additive noise at different SNR values (e.g., 10 dB, 5 dB, 0 dB),
- convolution with room impulse responses to simulate reverberation. To reproduce a practical scenario, we did not use white noise; instead, we used background noises from the augmentation noise set provided by GraFP.

Datasets. We initially adopted the `FMA_small` subset, as suggested in the GraFP setup, to ensure comparability with the reference framework. During a qualitative error analysis, however, we observed that the dataset contains repeated/near-duplicate tracks (identical audio appearing under different identifiers), which can artificially inflate ambiguity and make some "errors" hard to interpret.

Top 100 albums dataset. To avoid evaluating on data too close to the GraFP training distribution, we also built a separate evaluation collection composed of the songs from the *Top 100 albums of all time* [3]. This dataset contains 1185 songs for a total size of 49 GB. After indexing, the stored fingerprints occupy 2.86 GB for the Shazam pipeline (hash inverted index) and 1.8 GB for GraFP (embedding index).

4 Method

The two identification approaches are introduced in Section 2, which also reflects the starting point of this project. Here we focus on the parts that are specific to our implementation and experimental setup.

Both pipelines expose the same interface: (i) *indexing*, which processes each reference track and stores its fingerprints in a database/index; and (ii) *recognition*, which extracts fingerprints from a query segment and returns the best matching song together with a confidence score.

In particular, the Shazam-style pipeline was implemented from scratch following Wang's description, while GraFP was used directly as provided by the authors and integrated into the same indexing/retrieval workflow.

Compute resources. Experiments were conducted on an NVIDIA A30 (24 GB) GPU.

4.1 Shazam hyperparameter exploration

We performed an exploratory analysis by varying critical parameters of the Shazam-style pipeline, including the target sampling rate (`target_sr`: 8,000 / 11,025 / 16,000 Hz), FFT size (`n_fft`: 1024 / 2048 / 4096) and hashing fan-out (`fan_out`: 3 / 5 / 10). However, this grid search led to a drastic drop in recognition accuracy. The reason is intrinsic to the hashing scheme: the time difference Δt between peaks is encoded as a discrete number of spectrogram frames. Changing the time–frequency resolution (via sampling rate or FFT size) changes the physical duration of a frame, effectively producing incompatible hash tokens between the query and a previously built database.

Qualitatively, increasing `n_fft` (e.g., 2048→4096) improves frequency resolution but worsens time resolution (more stable for tonal content, typically less precise for transients), while decreasing `n_fft` (e.g., 2048→1024) has the opposite effect and can increase hash collisions due to coarser frequency bins. Changing `target_sr` shifts the frequency axis and the set of detectable peaks: higher rates preserve more high-frequency detail (at higher compute cost), while lower rates discard high-frequency content and can reduce distinctiveness.

A fully correct comparison would therefore require re-indexing the entire dataset for each configuration, which was deemed computationally too expensive for the scope and resources of this project.

4.2 Query-time optimizations for Shazam-style matching

Compared to the reference formulation by Wang, which can attempt to match every hash extracted from the query, we introduced a lightweight selection strategy to reduce computational load while keeping (and in some cases improving) discriminative power.

Rarity-based sampling. When a query produces a large number of hashes, we cap the number of processed tokens (e.g., up to a few hundred, scaled with query duration) and prioritise hashes that are rare in the database. This follows an information-retrieval intuition similar to inverse document frequency: very frequent hashes (appearing in many tracks) carry little discriminative information and tend to increase spurious matches, while rare hashes are more specific. Operationally, we estimate the posting-list size for each candidate hash and discard overly frequent tokens, focusing the search on distinctive fingerprints. To motivate this choice, Table 1 reports the timing

Wang step (clean, 30 s)	Time (s)
Extract spectrogram	5.0224
Find peaks	0.0139
Compute frequencies	0.0001
Build hashes	0.5603
Hash matching and voting	26.8184
Scoring preparation	0.1679
Entropy scoring	0.0020
Lookup	0.0026
Total recognition time	32.6523

Table 1: Per-query timing breakdown for the baseline Wang-style formulation (clean, 30 s). The matching stage retrieves 80,421,745 total matches.

breakdown of Wang's original formulation (processing all query hashes), where hash matching and voting dominates and produces tens of millions of candidate matches.

On-the-fly pruning in hash generation. We also reduced overhead during token construction by selecting target peaks (fan-out) in a single pass, avoiding expensive full-list sorting before pairing.

Adaptive query sizing. The maximum number of processed hashes is not fixed, but scales approximately linearly with query length (e.g., from about 200 hashes for 3 s up to about 1000 hashes for 15 s), ensuring comparable computational cost across conditions.

End-to-end engineering optimizations. Beyond query-time matching, we also profiled and optimized other pipeline components. As an example, we benchmarked audio resampling from 44.1 kHz to 16 kHz using a 180 s clip. Among the tested implementations, `torchaudio.functional.resample` on CPU (35.4 ms on average) and `soxr.resample` (38.1 ms) were about 3.6× and 3.4× faster than `scipy.signal.resample_poly` (128.5 ms), while GPU resampling was not consistently advantageous due to variance/overheads. These measurements informed our implementation choices to reduce preprocessing latency.

Overall, together with minor data-structure and code-level optimisations, these changes yielded an empirical speedup of about 63× with respect to our initial implementation.

4.3 Hash posting-list analysis (Shazam-style)

The following analysis was performed on the `FMA_small` subset. The Shazam-style inverted index stores, for each hash token, a *posting list* containing all occurrences of that token in the reference database (as pairs `(songID, anchorTime)`). To better understand both efficiency and discriminability, we analysed the empirical distribution of posting-list sizes and the degree to which each list is dominated by a single song.

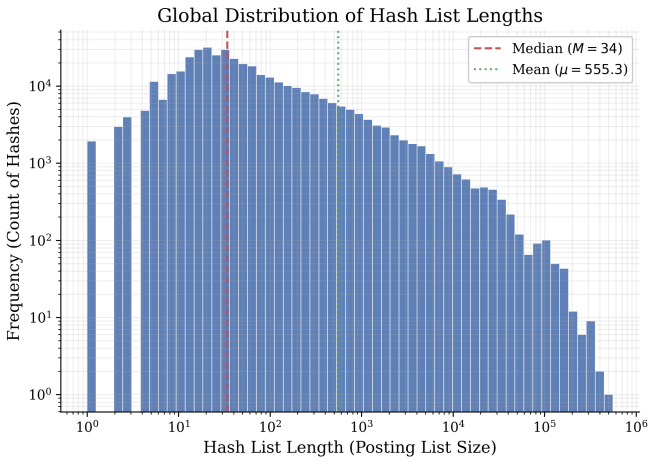

Figure 1: Histogram of posting-list lengths (entries per hash) in log–log scale. The distribution exhibits a pronounced long tail: most hashes are rare, while a small fraction generates very large lists.

Metric	Value
Unique hashes	387,743
Indexed songs	7,995
Total entries	215,318,839
Entries/song (mean)	∼26,932
List length (min/median/mean/max)	1 / 34 / 555 / 561,334
List length (P90/P99/P99.9)	545 / 9,669 / 57,417
Unique songs per list (median/mean/max)	18 / 95 / 6,406

Table 2: Summary statistics of the Shazam-style inverted index posting lists.

The gap between mean (555) and median (34) entries per hash confirms a highly skewed, heavy-tailed distribution (Figure 1). In practice, this implies that only a small subset of hashes is responsible for a disproportionate fraction of the retrieval workload, because common tokens return large candidate sets and increase the number of spurious matches.

To quantify whether large lists are mainly caused by a single track repeatedly generating the same token, we computed the *dominance ratio*: for each hash,

the fraction of its list occupied by the most frequent song. Across all hashes, the dominance is moderate on average (mean 18.84%, median 15.79%), but with a tail up to fully dominated lists (100%). In particular, 19,035 hashes (4.91%) have dominance $\geq 50\%$, while only 3,780 (0.97%) and 2,540 (0.66%) exceed 75% and 90%, respectively; 2,477 hashes (0.64%) are completely dominated.

Overall, these findings support our query-time design choice to prioritise *rare* hashes (short posting lists) during matching: they are computationally cheaper and typically more discriminative, whereas very common hashes contribute limited information while amplifying accidental collisions.

4.4 Confidence estimation (entropy-based)

We initially experimented with a softmax-based confidence computed from candidate scores, but it was consistently over-optimistic (often saturating close to 1). We therefore adopted an entropy-based metric, which is shared by both approaches.

Given vote counts v_i over the top-N candidate songs, we form a probability distribution

$$p_i = \frac{v_i}{\sum_{j=1}^{N} v_j}, \qquad H = -\sum_{i=1}^{N} p_i \log p_i,$$

and define the normalized confidence as

$$\text{Confidence} = 1 - \frac{H}{\log N}.$$

This yields high confidence when one candidate dominates the votes (low entropy), and low confidence when votes are spread across many candidates (high entropy).

Shazam-style votes. Votes are the height of the dominant peak in the offset histogram for each candidate track (i.e., the number of temporally consistent hash matches).

GraFP votes. Votes are the number of times a song appears among the k nearest neighbours retrieved (via FAISS) across all query embeddings.

5 Experiments and Results

5.1 Experimental protocol

We evaluate Top-1 identification accuracy, average query time, and average confidence. Queries are generated by taking 3 s, 5 s, and 10 s crops and applying (i) additive noise at different SNRs and (ii) convolutional reverberation via real impulse responses.

5.2 Shazam vs GraFP: results and comparison

Table 3 compares Shazam and GraFP in our experimental setup (500 queries).

Note on timing. The *Load audio* time includes file I/O plus preprocessing steps performed at query time (e.g., resampling to the target sample rate and applying audio augmentations).

Therefore, we exclude this step from the comparison since it is shared and identical for both methods.

Note on confidence. Although Table 3 reports a confidence value for both methods, these scores are not directly comparable across Shazam and GraFP because they are produced by different mechanisms (hash-voting vs embedding-neighbour aggregation). However, confidence remains informative *within* a single method when comparing conditions (e.g., clean vs SNR, or 10 s vs 5 s queries), as it reveals how the same model changes its certainty as the input becomes shorter or more degraded.

Note on accuracy. Accuracy in Table 3 was manually adjusted to account for copies with respect to the query in the dataset (e.g., repeated segments shared across original vs instrumental, remixes, or "feat." versions included in the same album).

5.2.1 Results commentary

The benchmark in Table 3 highlights a clear trade-off. On clean audio, Shazam achieves perfect accuracy and slightly outperforms GraFP (97.2–100.0%), but it is consistently slower (e.g., 459.5 ms vs 234.2 ms for 10 s queries). As distortions become stronger, GraFP is markedly more robust: at 0 dB SNR, GraFP reaches 95.4% accuracy vs 87.4% for Shazam; under reverberation (IR, 10 s) the gap widens to 98.0% vs 91.0%, and in the combined IR+SNR 5 dB condition to 97.4% vs 80.4%.

From the timing breakdowns (Tables 4–5), the dominant cost is the core matching/search stage. In Shazam, hash matching and voting is the largest cost (336 ms for clean 10 s), whereas in GraFP the FAISS search dominates the back-end stage (167 ms for clean 10 s) and model inference remains comparatively small (66 ms). Overall, the measured latencies suggest that both pipelines are feasible for interactive use, with GraFP providing the best robustness/latency balance in these experiments.

	Shazam			GraFP		
Condition	**Acc.**	**Time (ms)**	**Conf.**	**Acc.**	**Time (ms)**	**Conf.**
Clean (10 s)	**100.0%**	459.5	0.514	98.8%	**234.2**	0.681
Clean (5 s)	**100.0%**	256.9	0.369	98.0%	**109.6**	0.610
Clean (3 s)	**100.0%**	182.3	0.260	98.6%	**82.0**	0.544
SNR 10 dB	97.8%	449.3	0.396	**98.6%**	**235.2**	0.611
SNR 5 dB	97.4%	438.6	0.344	**98.6%**	**234.0**	0.560
SNR 0 dB	87.8%	422.9	0.269	**95.8%**	**234.4**	0.431
IR (10 s)	91.0%	445.8	0.123	**98.4%**	**233.6**	0.534
IR + SNR 5 dB	80.4%	407.1	0.085	**98.0%**	**233.3**	0.418

Table 3: Shazam vs GraFP on the benchmark (500 queries). *Load audio* (file I/O + identical preprocessing) was excluded from the timing comparison and takes about 0.53 s on average. Shazam indexes 1185 songs (DB load 27.31 s), GraFP 1174 songs (DB load 25.10 s).

Shazam step (clean, 10 s)	**Avg. time (ms)**
Extract spectrogram	2.2
Find peaks	4.6
Build hashes	98.5
Sample hashes	13.3
Hash matching and voting	336.0
Scoring preparation	1.3
Entropy scoring	0.2
Lookup	0.009
Total	459.5

Table 4: Per-query timing breakdown for Shazam (clean, 10 s).

GraFP step (clean, 10 s)	**Avg. time (ms)**
Transform	1.1
Model inference	65.8
Search	167.3
Total	234.2

Table 5: Per-query timing breakdown for GraFP (clean, 10 s).

5.2.2 Ablation study: effect of the query hash budget

A key implementation choice in the Shazam-style pipeline is the maximum number of query hashes processed during matching (`max_query_hashes`). This parameter trades off latency and robustness: a larger budget increases the number of retrieved postings (and thus compute), but can also strengthen the voting signal.

To quantify this effect, we performed an ablation study on the `top100` collection (1185 songs; ≈380k unique hashes; ≈381M total postings) using 10 clean queries of 10 s. We varied `max_query_hashes` from 50 to 2000, and also considered the unconstrained setting (`all`), which processes all hashes extracted from a query (on average ≈12,964 hashes). For each setting, we report Top-1 accuracy, the entropy-based confidence score defined in Section 4.4, and the per-query latency (excluding audio loading).

Max hashes	**Acc.**	**Conf.**	**Query (ms)**
50	100.0%	0.207	134
100	100.0%	0.289	129
200	100.0%	0.384	155
300	100.0%	0.448	188
500	100.0%	0.525	274
750	100.0%	0.561	430
1000	100.0%	0.577	635
1500	100.0%	0.608	1239
2000	100.0%	0.624	2241
all	100.0%	0.339	1,969,673

Table 6: Ablation on `max_query_hashes` for Shazam-style matching (10 clean queries, 10 s). The `all` setting processes all extracted hashes (on average ≈12,964), leading to prohibitive latency. Timing excludes audio loading.

Two trends are noteworthy. First, accuracy remains perfect across the tested range, indicating that a small subset of hashes is sufficient to identify these queries. Second, confidence increases steadily as more hashes are processed (up to 2000), but the unconstrained setting decreases confidence while dramatically increasing latency. This behaviour is consistent

with the presence of many frequent, low-information hashes: when all hashes are used, the match signal is diluted by spurious postings, while the back-end cost becomes dominated by large posting lists. Overall, this ablation supports the design choice of bounding the hash budget (typically 100–500 hashes) to obtain interactive latencies with strong confidence.

5.2.3 Scaling analysis: query time vs database size

We finally discuss how recognition latency is expected to scale with the database size N (number of indexed songs / fingerprints). In both systems, the query-time pipeline can be decomposed into (i) a *query-dependent* front-end (feature extraction and, for GraFP, neural inference) and (ii) a *database-dependent* back-end (lookup/search and aggregation).

Shazam. The Shazam-style pipeline comprises fingerprint extraction and hash-table matching. Fingerprint extraction (STFT, peak picking over the six bands, and hash construction) depends only on the query duration and the STFT configuration, and is therefore $\mathcal{O}(1)$ with respect to N. Database size matters primarily in the matching stage: each query hash h is used as a key into an inverted index that maps a 32-bit token to a posting list of $(songID, t_{\mathrm{db}})$ occurrences. The cost of matching is proportional to the total number of retrieved postings

$$T_{\mathrm{match}} \propto \sum_{i=1}^{Q} |\mathrm{bucket}(h_i)|, \tag{1}$$

where Q is the number of (sampled) query hashes. Let E be the total number of stored postings and H the number of unique hash keys. Under the simplifying assumption that hash frequencies are stable as we add more songs, the average posting-list length grows as E/H; since E increases roughly linearly with N while H tends to saturate (popular tokens are re-used across many tracks), the back-end stage is expected to grow approximately linearly with N. In practice, the constant factor is mitigated by amortised $\mathcal{O}(1)$ dictionary access and the rarity-based sampling, which caps Q and prioritises informative hashes.

Empirical scaling test (Shazam). To validate the discussion above, we ran an additional experiment by varying the number of indexed songs and measuring (i) average recognition time over 10 clean full-length queries (excluding audio loading), (ii) database size in terms of total postings (entries), (iii) number of unique hash keys, (iv) confidence range, and (v) database load time. The results are reported in Table 7.

Two observations are particularly relevant. First, the query time is essentially constant over the tested range: while the database grows by $\sim$10$\times$ in entries (33 M$\rightarrow$322 M), the average query latency increases by only about 100 ms ($\approx 2.5\%$). Notably, the theoretical $\mathcal{O}(N)$ dependence of the matching stage is not visible in the measured query latency within the tested range: the rarity-based sampling effectively transforms the linear back-end cost into a near-constant one by bounding the number of retrieved postings per query. During recognition, we sample a bounded number of query hashes (capped at 1000 for clips longer than 15 s) and preferentially select those with short posting lists. As a consequence, even if long collision lists become more common as N increases, the recogniser tends to avoid congested keys, keeping the effective number of retrieved postings per query roughly bounded.

Second, the number of unique hash keys saturates quickly (around 380 k): already at $N = 100$ songs the database covers most of the hash space observed in this collection. Thus, increasing N mostly increases posting-list lengths (entries) rather than creating many new keys. In this regime, confidence is a more sensitive indicator of scaling than accuracy: as N grows, spurious votes become more frequent, increasing entropy and reducing confidence, while Top-1 accuracy remained 100% in our test. Finally, the only metric showing a clear linear trend is the database load time, consistent with pickle deserialisation cost being proportional to the number of stored postings.

GraFP. GraFP consists of an audio transform, a neural forward pass, and a FAISS nearest-neighbour search. The transform and model inference depend only on the query input size and are $\mathcal{O}(1)$ with respect to N. The database-dependent term is the ANN search. With an IVF+PQ index (`IndexIVFPQ`), coarse quantisation compares the query embedding (here $d = 128$) to a fixed number of centroids ($n_{\mathrm{list}} = 256$), selecting $n_{\mathrm{probe}} = 20$ lists to scan; this stage is constant in N. The fine stage scans approximately a fraction $n_{\mathrm{probe}}/n_{\mathrm{list}}$ of the database, and each comparison costs $\mathcal{O}(\texttt{code\_sz})$ table lookups (here $\texttt{code\_sz} = 8$) rather than $\mathcal{O}(d)$ for brute force.

N songs	Query (ms)	Entries (M)	Unique hashes (k)	Conf. range	Load (ms)
100	3862	33	324.8	0.749–0.879	518
200	3876	64	356.3	0.750–0.872	5200
350	3954	112	372.0	0.732–0.856	7348
500	3911	158	377.5	0.740–0.844	10938
750	3937	239	379.6	0.619–0.828	19621
1000	3960	322	380.1	0.617–0.818	24143

Table 7: Shazam empirical scaling test (clean, full-length queries). *Entries* counts total posting-list elements (*songID*, *anchorTime*) pairs. The adaptive hash budget caps at 1000 hashes for clips longer than 15 s.

A simple proxy for the search cost per segment is

$$T_{\text{search}} \propto \frac{n_{\text{probe}}}{n_{\text{list}}} N \, \texttt{code\_sz}, \tag{2}$$

which is linear in N but with a substantially smaller constant than exhaustive search. For a 10 s query, multiple overlapping windows are embedded; if S segments are produced, the overall database-dependent term scales as $\mathcal{O}(S\,N)$. For small-to-medium collections, constant overheads (e.g. kernel launches, memory access) can dominate, and the linear trend becomes clearly visible only at larger scales.

6 Application Architecture

6.1 Backend Server (app.py)

The file `app.py` implements a FastAPI-based web server that serves as the core backend for the SongFinder application.

6.1.1 API Endpoints

`GET /` Serves the `songfinder.html` web interface.

`GET /health` Health check endpoint.

`GET /stats` Returns database statistics (total songs, fingerprints).

`POST /recognize` Main recognition endpoint that accepts audio files in multiple formats (MP3, WebM, WAV, M4A, OGG), converts them using FFmpeg if needed, and returns JSON with song title, confidence score, and updated cumulative votes.

6.1.2 Audio Processing Pipeline

Shazam: Accepts optional `cumulative_votes` parameter from previous queries and returns updated cumulative votes in the metadata.

GraFP: Resamples audio, converts to mono, generates embeddings, and performs FAISS-based similarity search.

6.2 Cumulative Voting System

The interface implements timed recognition at 3 s, 5 s, 10 s, and 15 s intervals. This allows the system to provide increasingly accurate results as more audio is captured while maintaining responsiveness.

The cumulative voting system enables **progressive confidence building** across multiple consecutive audio queries from the same recording session.

7 Discussion

Our benchmark confirms that landmark-based fingerprinting remains extremely effective on clean audio, where Shazam-style matching reaches near-perfect identification accuracy. However, under heavy distortions (strong noise and reverberation), the learned GraFP embeddings provide substantially better robustness, likely because the encoder is trained with augmentation pipelines that mimic these degradations.

From a systems perspective, query latency is dominated by the retrieval backend (hash matching and voting for Shazam; ANN search for GraFP). While Shazam can be highly accurate, GraFP achieves lower and more stable latencies across conditions in our setup.

8 Conclusion

We implemented a Shazam-style fingerprinting system from the original description and integrated the GraFP framework into a unified indexing and recognition pipeline. On the evaluated dataset (around 1.2k songs) and 500 queries per condition, Shazam is slightly better on clean audio, whereas GraFP is markedly more robust to reverberation and extreme noise while also being faster.

References

[1] A. Wang, *An Industrial Strength Audio Search Algorithm*, in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2003.

[2] A. Bhattacharjee, S. Singh, and E. Benetos, *GraFPrint: A GNN-Based Approach for Audio Identification*, in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5, 2025.

[3] Rolling Stone, *The 500 Greatest Albums of All Time*, `https://www.rollingstone.com/music/music-lists/best-albums-of-all-time-1062063/`, accessed 2026-02-06.